

# Essentials of Data Science Laboratory - 2304102L - 2026 - 2304102L

Search course ctrl + k

- 1. Practical 1
- 2. Practical 2
- 3. Practical 3
- 4. Practical 4
- 5. Practical 5



Pickup from where you left off!

## Scatter Plot for Age vs. Fare by Survived

Practical 5 • Lab Assignment

**Resume**

**Practical 1**

Unit • 100% completed

Progress bar: 100%

[>](#)

**Practical 2**

Unit • 100% completed

Progress bar: 100%

[>](#)

**Practical 3**

Unit • 100% completed

Progress bar: 100%

[>](#)

# Essentials of Data Science Laboratory - 2304102L - 2026 - 2304102L

Dashboard **Contents** Calendar Assessments Syllabus

Search course ctrl + k

- 1. Practical 1
- 2. Practical 2
- 3. Practical 3
- 4. Practical 4
- 5. Practical 5

Unit - 100% completed

**Practical 2**

Unit - 100% completed

**Practical 3**

Unit - 100% completed

**Practical 4**

Unit - 100% completed

**Practical 5**

Unit - 100% completed

1.1.1. Calculate Momentum

11:49

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum  $p$  is calculated using the formula:

$$p = m \times v$$

where:  
 $m$  is the mass of the object (in kilograms).  
 $v$  is the velocity of the object (in meters per second).

- Input Format:**
- A single floating-point number representing the mass of the object in kilograms.
  - A single floating-point number representing the velocity of the object in meters per second.

- Output Format:**
- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases

calculate...

Submit

Debugger

1 #Type Content here...  
2 m=float(input())  
3 v=float(input())  
4 p=m\*v  
5 print(f"{p:.2f}kgm/s")

Average time  
0.008 s  
8.00 ms

Maximum time  
0.009 s  
9.00 ms

2 out of 2 shown test case(s) passed  
2 out of 2 hidden test case(s) passed

Test case 1 9 ms Debug

Expected output  
5.0  
10.0  
50.00kgm/s

Actual output  
5.0  
10.0  
50.00kgm/s

Test case 2 7 ms

Terminal

Test cases

1.1.2. Conditional Calculation Based on the Number of Digits

08:42

Write a Python program that accepts an integer  $n$  as input. Depending on the number of digits in  $n$ .

**Constraints:**  
 $1 \leq n \leq 999$

**Input Format:**  
• The input consists of a single integer  $n$ .

**Output Format:**  
• If  $n$  is a single-digit number, print its square.  
• If  $n$  is a two-digit number, print its square root (rounded to two decimal places).  
• If  $n$  is a three-digit number, print its cube root (rounded to two decimal places).  
• Else print "Invalid".

Sample Test Cases

+

condition...

1 #write your code here...  
2 n=int(input())  
3 if 1 <= n <= 9 :  
4     print(n\*\*2)  
5  
6 elif 10 <= n <= 99:  
7     print(format(n\*\*0.5, ".2f"))  
8  
9 elif 100 <= n <= 999:  
10     print(format(n\*\*(1/3), ".2f"))  
11

Average time  
**0.006 s**  
6.00 ms

Maximum time  
**0.007 s**  
7.00 ms

4 out of 4 shown test case(s) passed  
3 out of 3 hidden test case(s) passed

Test case 1 7 ms Debug

Expected output  
9  
81

Actual output  
9  
81

Test case 2 5 ms

Test case 3 5 ms

Terminal Test cases

### 1.1.3. Days between Two Dates

Write a Python program to calculate number of days between two dates.

#### Input Format:

- The first line contains the first date in the format *YYYY – MM – DD*.
- The second line contains the second date in the format *YYYY – MM – DD*.

#### Output Format:

- An integer representing the number of days between the given dates.

#### Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases

```
noOfDay...  
1 from datetime import date  
2 y1,m1,d1=map(int,input().split('-'))  
3 y2,m2,d2=map(int,input().split('-'))  
4 date1 = date(y1,m1,d1)  
5 date2 = date(y2,m2,d2)  
6 diff=(date2-date1)  
7 print(diff.days)
```

Average time  
**0.008 s**  
8.25 ms

Maximum time  
**0.009 s**  
9.00 ms

✓ 2 out of 2 shown test case(s) passed  
✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 **9 ms** Debug

Expected output

2014-07-02

2014-11-02

123

Actual output

2014-07-02

2014-11-02

123

✓ Test case 2 **8 ms**

Terminal

Test cases

< Prev Reset Submit Next >

1.1.4. Reverse a Number

19:53

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

The input is a single integer.

Output Format:

The output prints a single integer, which is the reversed number.

Sample Test Cases

reverseN...

Submit

1 num = int(input())  
2 reversed\_num = int(str(num)[::-1])  
3 print(reversed\_num)

Average time  
0.005 s  
5.00 ms

Maximum time  
0.006 s  
6.00 ms

2 out of 2 shown test case(s) passed  
3 out of 3 hidden test case(s) passed

Test case 1 6 ms

Debug

Expected output  
5367  
7635

Actual output  
5367  
7635

Test case 2 5 ms

Terminal

Test cases

1.1.5. Multiplication Table

02:33

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

Print the multiplication table for the given number in the format:

<number> x <multiplier> = <result>

Note:

The character "x" is used in the output to represent multiplication.  
Refer to the visible test-cases for more clarity.

Sample Test Cases

multipl...

1 #write your code here...

2

3

4 num = int(input())

5

6 for i in range(1,11):

7 result = num\*i

8 print(f"{num} x {i} = {result}")

Average time0.005 s5.50 ms

Maximum time0.007 s7.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 17 ms

Expected output

Actual output

8

8 x 1 = 8

8 x 2 = 16

8 x 3 = 24

8 x 4 = 32

8 x 5 = 40

Terminal

Test cases

### 1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

#### Input Format:

- The first input will be an integer  $n$ , the number of courses.
- The second input will be  $n$  integers representing the marks of the student in each of the  $n$  courses, separated by a space.

#### Output Format:

- If the student passes all courses:
- Print the aggregate percentage (formatted to two decimal places).
  - Print the grade based on the aggregate percentage.
- If the student fails any course (marks < 40 in any course), print:
- "Fail".

#### Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

passorFa...

1

# write your code here

2

n = int(input())

3

marks = list(map(int, input().split()))

4

5

6

if all(mark >= 40 for mark in marks):

7

a = sum(marks) / n

8

print(f"Aggregate Percentage: {a:.2f}")

9

if a > 75:

10

print("Grade: Distinction")

11

elif 60 <= a < 75:

Average time

0.012 s

12.10 ms

Maximum time

0.032 s

32.00 ms

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test case 1 11 ms Debug

Expected output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Actual output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Test case 2 8 ms

Terminal Test cases

1.2.2. Fibonacci Series

03:18

Write a Python program that uses recursion to print the first  $n$  terms of the Fibonacci series.

Input Format:

- A single integer  $n$  representing the number of terms to generate.

Output Format:

- A single line containing the first  $n$  terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

fibonacci...

def fibonacci(n):  
 # write the code..  
 if n == 1:  
 return 0  
 if n == 2:  
 return 1  
 else:  
 return fibonacci(n-1)+fibonacci(n-2)

Average time  
0.013 s  
12.75 ms

Maximum time  
0.032 s  
32.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 6 ms Debug

Expected output  
5  
0 1 1 2 3

Actual output  
5  
0 1 1 2 3

Test case 2 5 ms

Terminal

Test cases

Prev

Reset

Submit

Next

1.2.3. Pattern - 1

00:35

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (\*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

rightangl...

1 #Type Content here...  
2  
3 n=int(input())  
4 for i in range(1,n+1):  
5 print('\*'\*i)

Average time0.008 s  
8.33 ms

Maximum time0.013 s  
13.00 ms

2 out of 2 shown test case(s) passed  
4 out of 4 hidden test case(s) passed

Test case 113 ms

Debug

Expected output

Actual output

5

\*  
\* \*  
\* \* \*  
\* \* \* \*  
\* \* \* \* \*

5

\*  
\* \*  
\* \* \*  
\* \* \* \*  
\* \* \* \* \*

Terminal

Test cases

1.2.4. Pattern - 2

00:39

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases

numberP...

Submit

1 # Type Content here...

2

3 n = int(input())

4 for i in range(1,n+1):

5 for j in range(1,i+1):

6 if i==j:

7 print(j,end=' ')

8 else:

9 print(j,end=' ')

10 print()

11

Average time

0.005 s

5.50 ms

Maximum time

0.007 s

7.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 7 ms

Debug

Expected output

5

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

Actual output

5

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

Terminal

Test cases